MONGODB — ARCHITECTURE, OPERATIONS & REFERENCE V1

The document model, the consistency controls, and the two decisions that are hard to reverse
schema design · replica sets · read & write concern · sharding · indexing · aggregation · operations · **depth marked** junior → staff

1 · WHAT MONGODB IS — THE DOCUMENT MODEL, AND WHERE IT SITS ON CONSISTENCY

start here — what "flexible schema" actually means, and the two decisions you will struggle to undo

The definition

A **document database**. Records are BSON documents — a binary encoding of JSON with additional types — grouped into collections. A document holds nested objects and arrays, so an entire aggregate can be stored and retrieved as **one record, with no join**.

"Flexible schema" does not mean "no schema". The schema still exists — it has moved from the database into your application code, where nothing enforces it unless you make it. Every consumer must handle every historical shape a document has ever had. This is a genuine trade, not a free advantage, and treating it as an absence of schema is how document databases acquire their reputation.

Hard limits worth memorising

16 MB per document	A hard BSON limit. It is the reason unbounded arrays are the classic failure mode
100 levels of nesting	Rarely reached, but real
Index key ~1024 bytes	Long strings cannot be fully indexed
Shard key is near-permanent	Resharding exists (5.0+) but is a heavy operation — treat the choice as one-way

The two decisions that are hard to undo — and therefore where the thinking belongs: **the document model** (what is embedded versus referenced) and **the shard key**. Almost everything else on this sheet is tunable later; these two are structural.

Where it sits on CAP

MongoDB is CP by default, not AP. A replica set has exactly **one primary** that accepts writes. During a partition, the minority side has no primary and therefore **cannot accept writes** — it sacrifices availability to preserve consistency. It is frequently mislabelled AP alongside Cassandra, and the two behave quite differently.

But it is tunable, which is the real answer. Write concern and read concern let you move along the spectrum **per operation** — from fast and loss-prone to majority-acknowledged and linearizable. The classification follows the configuration, not the product. Panel 6 is where this is set.

Use it when

- Data is naturally **document-shaped** — an aggregate read and written as a unit
- The schema **varies by record** or evolves quickly — product catalogues, content, events
- You need **horizontal write scaling** and can choose a good shard key
- Queries are mostly **by key or by indexed field on one collection**
- Geospatial, time-series, or catalogue workloads where the flexibility is genuinely used

Do not use it when

- **The data is genuinely relational** — many-to-many across several entities, queried from every direction. You will end up doing joins in application code and hand-maintaining duplicated fields
- **You need multi-table transactions constantly.** They exist, but the model is designed so you rarely need them; needing them often is a signal the schema is wrong
- **Ad-hoc analytical queries over history** — a columnar engine is the right tool
- **Strong referential integrity is required** across collections — there are no foreign keys
- **The working set will not fit in RAM** and the access pattern is random — performance falls off sharply

The honest failure mode. Most unhappy MongoDB deployments are relational data modelled as documents. The symptom is recognisable: joining three collections in application code, keeping duplicated fields in sync by hand, and reaching for transactions to make it safe. At that point a relational database has been rebuilt without its guarantees.

STAFF **The reframe worth offering.** The question is never "document or relational" in the abstract — it is **"does one document correspond to one unit of work?"** If reads and writes naturally touch one aggregate, the model fits and everything is easy. If they routinely span several, it does not, and no amount of tuning compensates.

2 · THE DOCUMENT MODEL — EMBED OR REFERENCE

the decision that determines whether everything else is easy · JUNIOR the rule · STAFF the cardinality cases

The governing rule, in one line: data that is accessed together should be stored together. Model from your *queries*, not from your entities. This is the exact inverse of relational design, where you normalise first and let the query planner assemble.

```
// EMBEDDED — one read, no join
{ _id: 1, name: "Ada",
  address: { city: "Bengaluru", pin: "560001" },
  recentOrders: [ {id: 9, total: 2499}, … ]   ← bounded
}

// REFERENCED — separate collection, joined by app or $lookup
{ _id: 1, name: "Ada" }
{ _id: 9, userId: 1, total: 2499 }            ← unbounded
```

Choosing by cardinality

Relationship	Model as
One-to-one	**Embed.** An address inside a user — always read together, never separately
One-to-few tens, bounded	**Embed.** Order lines, phone numbers, tags. The bound is what makes it safe
One-to-many hundreds, growing	**Reference**, with the child holding the parent id. Optionally embed a small recent subset for display
One-to-very-many unbounded	**Always reference.** Comments on a post, events for a device, log lines
Many-to-many	Reference from both sides, or accept duplication with a plan for keeping it in step

The single most common MongoDB mistake: the unbounded array. Embedding a collection that grows without limit — comments, events, readings, followers — fails in three ways at once. The document eventually approaches the **16 MB ceiling** and writes start failing. Every update **rewrites the whole document**, so cost grows with array length. And the document outgrows its allocated space repeatedly, causing churn. It works perfectly in testing and fails months later in production.

The trade, stated plainly

	Embed	Reference
Reads	One operation, no join	Two or more, or a `$lookup`
Writes	Atomic across the whole aggregate, for free	Multiple documents — may need a transaction
Growth	**Bounded by 16 MB**	Unbounded
Duplication	Data repeats across documents	Single source of truth
Update cost	Changing shared data means touching many documents	Change once

Atomicity is the underrated reason to embed. A write to a single document is **always atomic**, however deeply nested the change. Embedding an aggregate means the invariants inside it are maintained without transactions — which is the model working as intended, and why needing transactions frequently suggests the boundaries are wrong.

Duplication is acceptable here, deliberately. Storing an author's name alongside each post so the post renders in one read is normal practice, not a normalisation failure. What it costs you is a job to update those copies when the name changes — decide up front whether that is acceptable, and how stale a copy may be.

3 · SCHEMA PATTERNS

named solutions to recurring shapes

Pattern	Use when
Bucket	High-volume time-series or events. Group many readings into one document per device per hour, instead of one document per reading. **Dramatically fewer documents and smaller indexes**
Computed	A value is read far more often than its inputs change. Store the total, average or count rather than recomputing on every read
Subset	Only a small part of a large related set is usually needed. Embed the ten most recent, reference the rest
Extended reference	A join is needed only for a couple of fields. Duplicate just those alongside the reference, and accept the update cost
Polymorphic	Similar entities with differing fields share a collection, distinguished by a type field. This is the flexible schema being used deliberately
Attribute	Many optional, rarely-queried fields. Store as an array of key/value pairs so **one index covers them all**
Outlier	Most documents are small but a few are enormous — a normal user versus a celebrity. Flag the outliers and handle them on a separate path
Approximation	Counters that do not need exactness — increment every *n*th event rather than every one

Anti-patterns to recognise

- **Unbounded arrays —** the one that causes real outages
- **Bloated documents —** fetching 2 MB to read one field
- **A collection per tenant or per user —** thousands of collections carry real metadata and memory overhead
- **Separating data always read together —** relational habit applied to a document store
- **Indexing everything —** each index slows every write and consumes RAM
- **Case-insensitive queries** without a collation or a normalised field — a guaranteed collection scan

The bucket pattern deserves attention if you are storing anything time-series shaped. It reduces document count by orders of magnitude, and MongoDB's native **time series collections** (5.0+) implement essentially this idea for you, with automatic bucketing and better compression.

4 · ARCHITECTURE — ONE NODE, A REPLICA SET, AND A SHARDED CLUSTER

what runs inside mongod, and the two topologies — each solving a different problem

INSIDE ONE mongod — WiredTiger is where the durability and concurrency behaviour actually lives

Clients / drivers
official drivers per language
connection pooling
retryable reads and writes
topology-aware
TCP 27017

→ **Query layer**
· Parse and validate the command
· Query planner — chooses an index or a scan
· Plan cache, with periodic re-evaluation
· Aggregation pipeline execution
A cached plan can go stale and slow a query

→ **WiredTiger storage engine**
· **Document-level concurrency** — writers do not block each other across different documents
· **MVCC** — readers see a consistent snapshot
· Compression on by default for collections
B-tree structures on disk, cached in memory

→ **Cache — the performance story**
· Default size: **50% of (RAM − 1 GB)**, minimum 256 MB. Set it explicitly in containers, where host RAM may be detected instead of the limit
· Holds working set: documents and index pages
Working set not fitting here is the main slowdown

Durability — journal and checkpoints
· **Journal** — a write-ahead log, flushed frequently. Recovery replays it
· **Checkpoints** — a consistent on-disk snapshot written periodically
· A crash loses at most the window since the last journal flush
· `j: true` on a write waits for the journal before acknowledging
Journalling protects against process crash — not against losing the host

HOW A WRITE ACTUALLY COMPLETES — and where each guarantee is established
The document is modified in the WiredTiger cache → an entry is appended to the journal → an entry is appended to the **oplog**, which is itself a capped collection → secondaries tail the oplog and apply the change → a checkpoint eventually writes the data files. **When the client is told the write succeeded depends entirely on the write concern** — see panel 6.
With the weakest concern the client is acknowledged before any of the durability steps complete. That is a valid choice, and it must be a deliberate one.

THE OPLOG — the mechanism behind replication, change streams and point-in-time recovery
A **capped collection** of idempotent operations. Secondaries tail it and replay; change streams read it; point-in-time restore replays it over a snapshot.
Its size determines the **replication window** — how long a secondary may be offline and still catch up incrementally.
If a secondary falls behind the oldest oplog entry it can no longer resync incrementally and needs a full initial sync — expensive, and it happens exactly when the cluster is already under strain.

TWO TOPOLOGIES — they solve different problems, and a sharded cluster is built out of replica sets

REPLICA SET — availability and read scaling

PRIMARY all writes go here → **SECONDARY** tails the oplog → **SECONDARY** can be elected

· **Odd number of voting members** — 3 or 5
· A majority must be reachable to elect a primary
· Automatic failover, typically in seconds

· **Exactly one primary.** Writes never scale by adding members — only reads do, and only if you accept stale data
· An **arbiter** votes without holding data. It makes a two-member set electable, and it means a majority write concern cannot be satisfied by data-bearing nodes alone — which is why arbiters are discouraged for anything that matters
· **Members should sit in separate failure domains, or the majority can be lost to a single event**

SHARDED CLUSTER — horizontal scale of data and writes

mongos router — stateless | **Config servers** their own replica set | **Shard 1** a replica set | **Shard N** also a replica set

· **mongos** routes each query using the cluster metadata. It holds no data and is scaled horizontally beside the application
· **Config servers** hold the authoritative chunk-to-shard map. They are themselves a replica set, and losing them is fatal to routing
· Data is split into **chunks** by shard key range; the **balancer** moves chunks to keep shards even
· **Every shard is a replica set** — sharding gives capacity, replication gives availability. You need both
· **Do not shard until you must: it adds routing, balancing, and query constraints that a replica set does not have**

FAILOVER AND ROLLBACK — the behaviour that surprises people
When a primary becomes unreachable, the remaining members hold an election. A majority must agree, which is why an odd number in separate failure domains matters.
A secondary is promoted, typically within seconds, and drivers discover the new primary automatically.
The consequence: writes that were acknowledged but had not replicated to a majority can be ROLLED BACK. The old primary, on rejoining, writes them to a rollback file and discards them from the live data. The client was told they succeeded, and they are gone.
This is the entire reason write concern exists as a per-operation control. Writing with `w: "majority"` is what makes a write survive a failover — see panel 6.

READ ROUTING — read preference decides which member answers, and therefore how stale the answer may be

THE FIVE READ PREFERENCES
· **primary** — the default. Always current, and no read scaling
· **primaryPreferred** — primary, falling back to secondaries
· **secondary** — secondaries only. Scales reads, may be stale
· **secondaryPreferred** — secondaries, falling back to primary
· **nearest** — lowest latency, regardless of role
Tag sets and maxStalenessSeconds refine which members qualify

READING FROM SECONDARIES IS NOT FREE READ SCALING
· Replication is asynchronous, so a secondary read can return **arbitrarily stale** data during lag
· A user who writes and immediately reads may not see their own change
· Secondaries apply the same write load as the primary, so they are not idle capacity
· `maxStalenessSeconds` bounds it, but does not eliminate it
Use secondary reads for analytics, reporting and genuinely stale-tolerant paths — not as a general answer to a primary that is running hot.

CAUSAL CONSISTENCY — the fix for read-your-writes
A **causally consistent session** guarantees that operations within it observe a consistent order — so a read after your own write sees that write, even on a secondary.
The driver carries a cluster time and an operation time on each request, and the server waits until the member is current enough to satisfy it.
This is the correct answer to "I read from a secondary and my own update was missing" — rather than moving the read back to the primary and losing the scaling.

QUERY ROUTING IN A SHARDED CLUSTER — the distinction that decides whether sharding helps at all
Targeted query — the filter includes the shard key, so mongos routes to exactly the shard or shards holding those chunks. This scales: adding shards adds capacity, and each query touches few of them.
Scatter-gather — the filter omits the shard key, so mongos must ask **every shard** and merge the results. Latency becomes that of the slowest shard, and adding shards makes it worse rather than better.
Sorting without the shard key, and paginating deeply, both amplify this — mongos must gather and merge far more than it returns.
This is why the shard key is chosen from the query patterns, not from the data model. A shard key that no common query filters on gives you all the operational cost of sharding and none of the scaling — and it is the single most expensive mistake available in a MongoDB deployment.

READING THE DIAGRAM — the four properties that explain the rest of this sheet

1 · One primary accepts all writes. Replication gives availability and read capacity, never write capacity. When writes exceed one primary, the answer is sharding — and sharding brings the routing constraints above.

2 · Consistency is a per-operation choice, not a property of the product. Write concern decides when you are told a write succeeded; read concern decides what a read is allowed to see. MongoDB is CP by default and can be tuned in both directions.

3 · The working set should fit in the WiredTiger cache. Performance is excellent while it does and degrades sharply when it does not, because every miss becomes disk I/O. Capacity planning here is memory planning first.

4 · A single-document write is always atomic. Model so that one document is one unit of work and you rarely need transactions. Needing them frequently is a signal about the schema rather than a reason to tune the transaction.

Useful interrogation of any MongoDB design: what does one document represent, does any array in it grow without bound, what is the shard key and which queries filter on it, and what write concern is the application actually using?

Those four questions surface almost every serious problem — and the last one is very often answered with "whatever the default is", which is worth knowing before it matters.

5 · REPLICA SETS — ELECTIONS, THE OPLOG, AND WHAT SURVIVES A FAILOVER

MID how failover works · STAFF rollback, oplog window and the arbiter trap

Membership and elections

- **An odd number of voting members —** three or five. An even number gains no fault tolerance and can deadlock a vote
- **A majority must be reachable** to elect a primary. Three members tolerate one failure; five tolerate two
- **Members in separate failure domains —** three members in one rack protect against a process crash and nothing else
- `priority` influences who is elected; setting it to 0 makes a member ineligible — useful for a distant or analytics-only node
- **Hidden and delayed members** serve backups or provide a human-error safety window

The arbiter trap. An arbiter votes but holds no data, letting two data-bearing members form a majority cheaply. But if one of those two fails, **a majority write concern can no longer be satisfied** — writes stall or lose their durability guarantee while the set still reports as healthy. Prefer a third data-bearing member for anything that matters.

The oplog

- A **capped collection of idempotent operations**, replayed by secondaries
- Its size sets the **replication window** — how long a member can be offline and still catch up
- Fall behind the oldest entry and a **full initial sync** is required, which is expensive and happens under strain
- **Size it against your write rate and your worst realistic maintenance window**, then monitor the window as a metric

Rollback, stated plainly. If a primary accepts writes that have not reached a majority and then fails, a new primary is elected without them. When the old primary rejoins, it **discards those writes** — they are written to a rollback file for manual recovery, and removed from the live data. The client was told they succeeded.

What prevents it

`w: "majority"`	The write is acknowledged only once a majority hold it, so an election cannot lose it. **The default since MongoDB 5.0**, and worth confirming rather than assuming in an older or overridden deployment
`j: true`	Waits for the journal, protecting against a process crash on the acknowledging members
Read concern `majority`	Reads only data that cannot be rolled back — the read-side counterpart

The trade is latency. A majority write concern waits for network round trips to other members, so it is slower than `w: 1` — noticeably so across regions. That is the actual cost of durability here, and it should be a considered choice per operation rather than a global default nobody revisits.

Operating a replica set

- **Rolling maintenance —** upgrade secondaries first, then step down the primary deliberately
- `rs.stepDown()` for a controlled failover rather than waiting for detection
- **Monitor replication lag —** it is the size of your data-loss window right now
- **Test failover deliberately**, including that the application reconnects. An untested failover is the most common reason recovery does not work when needed

6 · WRITE & READ CONCERN

the actual consistency controls

This panel is the consistency story. Every claim about MongoDB being consistent or eventually consistent reduces to these two settings — and they are chosen **per operation**, not per cluster.

Write concern — when am I told it succeeded?

Setting	Meaning
`w: 0`	Fire and forget. No acknowledgement at all — the write may simply not happen
`w: 1`	The primary has it. **Can be rolled back** if the primary fails before replicating
`w: "majority"`	A majority of data-bearing members have it. **Survives failover.** The default since 5.0
`w: N`	A specific count of members
`j: true`	Waits for the journal — durable against process crash
`wtimeout`	A time bound. **On timeout the write is not undone** — it may still be applied. This is not a rollback

Read concern — what am I allowed to see?

Level	Returns
`local`	The member's most recent data. **May be rolled back later**
`available`	Like local, with fewer guarantees in a sharded cluster — can return orphaned documents
`majority`	Only data acknowledged by a majority. **Will not be rolled back**
`linearizable`	The strongest. **Primary only**, and only for a query that uniquely identifies a single document. Confirms with other members, so it is measurably slower
`snapshot`	A consistent point-in-time view across the operation — used by multi-document transactions

The combination that catches people out. Writing with `w: "majority"` and then reading with `local` from a secondary can still return stale or rollback-able data. **Both sides must be set** — the write concern alone does not make reads safe.

7 · SHARDING — CHUNKS, THE BALANCER, AND WHAT CHANGES FOR YOUR QUERIES

MID the mechanism · STAFF when to shard, and what stops working

The mechanism

- A sharded collection is split into **chunks** — contiguous ranges of shard-key values
- The **config servers** hold the authoritative map of chunk to shard. They are a replica set, and their loss stops routing entirely
- **mongos** is a stateless router: it reads the map and directs each operation. Run several, close to the application
- The **balancer** migrates chunks between shards to keep the distribution even. Migration is real data movement and competes with live traffic
- **Every shard is itself a replica set** — sharding gives capacity, replication gives availability

Sharding strategies

Strategy	Behaviour
Ranged	Chunks by key range. **Range queries are efficient**, and a monotonically increasing key sends every insert to one shard
Hashed	Chunks by hash of the key. **Even write distribution**, and range queries become scatter-gather
Zoned	Pin key ranges to specific shards — for data residency, or tiering by hardware

The choice is a genuine trade. Hashed solves the hot-shard problem and gives up efficient range scans. Ranged preserves locality and risks a hotspot. A **compound shard key** — a well-distributed prefix followed by a range component — often captures both, and is usually the better answer.

What changes once a collection is sharded

Concern	Consequence
Queries without the shard key	Become scatter-gather across every shard, with latency set by the slowest
Unique indexes	Only enforceable if they include the shard key — you cannot cheaply guarantee global uniqueness on another field
Sorting and deep pagination	mongos must merge across shards; skipping deeply is expensive
`$lookup`	Historically restricted against sharded collections; capability varies by version. **Verify against your version rather than assuming**
Transactions	Work across shards from 4.2, at a higher coordination cost
Balancer activity	Migrations add load and latency. Schedule a balancing window if traffic has a clear peak

Do not shard prematurely. Sharding is the answer to *capacity* — data or write volume exceeding one replica set. It is not a performance fix for slow queries, and it is not a substitute for an index. Applied to a workload that did not need it, it adds routing, balancing and every constraint above while making the common queries slower.

Reach for it when the dataset exceeds what one machine can hold with the working set in RAM, or write throughput exceeds one primary. Before that, a larger machine and better indexes are almost always the cheaper answer.

8 · THE SHARD KEY

the decision to get right first

Treat this as one-way. Resharding a live collection is supported from MongoDB 5.0, but it copies the entire collection and is a major operation. Choosing well at the start is dramatically cheaper than fixing it later — and a bad shard key is the most expensive mistake available here.

The three properties it needs

Property	Why, and what goes wrong without it
High cardinality	Enough distinct values to split into many chunks. A key with few values produces **jumbo chunks** that cannot be divided and cannot be balanced
Even frequency	No single value should dominate. A key where 40% of documents share one value concentrates that data and its traffic on one shard permanently
Non-monotonic	A steadily increasing key — a timestamp, an ObjectId — sends **every new insert to the same shard**, because they all fall in the highest chunk range. The cluster has many shards and one of them is doing all the work
Present in queries	The fourth, and the one most often forgotten. If common queries do not filter on it, every one is scatter-gather and sharding buys nothing

```
// poor: monotonic — all inserts hit one shard
{ createdAt: 1 }

// poor: low cardinality — jumbo chunks, no balance
{ country: 1 }

// better: hashed — even writes, no range locality
{ userId: "hashed" }

// often best: compound — distributes AND targets
{ tenantId: 1, createdAt: 1 }
// queries filtering tenantId are targeted;
// within a tenant, time ranges stay local
```

The test to apply before committing. Write down your five most frequent queries. If the shard key does not appear in the filter of most of them, it is the wrong key — regardless of how evenly it distributes the data.

9 · INDEXING

and the rule for compound order

Type	For
Single field	Equality, range and sort on one field. `_id` is always indexed and unique
Compound	Several fields. **Order matters — see below**
Multikey	Created automatically on an array field; indexes every element
Text	Basic full-text. Limited — for real search, use a dedicated search index
Geospatial 2dsphere	Proximity and containment queries on coordinates
Hashed	Even distribution for sharding; equality only, no ranges
TTL	Deletes documents after an interval. A background task, so removal is not instant
Partial	Indexes only documents matching a filter — smaller and cheaper
Sparse	Skips documents lacking the field
Unique	Enforces uniqueness. On a sharded collection it must include the shard key
Wildcard	Indexes unknown or highly variable field names — useful with truly polymorphic documents

The ESR rule for compound index order: Equality, Sort, Range. Fields matched exactly come first, then fields used for sorting, then range conditions. Getting this order wrong is the most common reason an index exists and the query still sorts in memory or scans.

The rules that follow

- **A compound index serves any prefix of itself** — an index on `(a,b,c)` helps queries on `a`, or `a,b`, but not on `b` alone
- **Covered queries** answer entirely from the index without touching a document — the fastest possible read
- **Every index slows every write** and consumes RAM. Find and drop unused ones using index usage statistics
- **Indexes should fit in cache** alongside the working set, or lookups become disk reads
- **Build in the background** on production, and prefer a rolling build across members

The fastest diagnosis: `explain("executionStats")`. `COLLSCAN` means no index was used. A large gap between documents examined and documents returned means the index is not selective enough. Both are visible in seconds and are the first thing to check.

10 · QUERIES & AGGREGATION

the pipeline, and its limits

The aggregation pipeline passes documents through ordered stages, each transforming the stream. It is the mechanism for grouping, joining, reshaping and computing — everything beyond a simple find.

```
db.orders.aggregate([
  { $match:  { status: "shipped" } },   ← FIRST
  { $lookup: { from: "users", … } },
  { $group:  { _id: "$region",
               total: { $sum: "$amount" } } },
  { $sort:   { total: -1 } },
  { $limit:  10 }
])
```

The one optimisation that matters most: filter early. Put `$match` and `$limit` as early as possible. A `$match` at the head can use an index; the same stage after a `$group` cannot, and every prior stage has already processed documents you were going to discard.

Limits worth knowing

- **Stages have a memory limit**; exceeding it errors unless `allowDiskUse` is enabled, which then spills to disk and is much slower
- **A returned document still cannot exceed 16 MB —** a `$group` accumulating a large array will hit it
- **`$lookup` is not a relational join.** It is a left outer join executed per input document, with no query planner optimising across the two collections. Fine for small lookups, poor as a substitute for a proper data model
- **Sorting without a supporting index** happens in memory, subject to the same limit

If you find yourself writing several `$lookup` stages, that is a strong signal the documents are modelled relationally. The fix is in panel 2, not in the pipeline — no amount of aggregation tuning compensates for a schema that requires assembling every read.

Change streams expose the oplog as a resumable, filterable event stream — the supported way to react to data changes without polling. They require a replica set, and the resume token is what makes them safe across a consumer restart.

11 · TRANSACTIONS

available, and best avoided

The correction first, because the outdated claim persists. MongoDB has supported **multi-document ACID transactions since 4.0** for replica sets, and **4.2** extended them across sharded clusters. They are a mature capability, not a missing one. "MongoDB does not have transactions" has been wrong for years.

```
session = client.startSession()
session.withTransaction(() => {
  orders.insertOne({…},    { session })
  inventory.updateOne({…}, { session })
})   // commits or aborts together
```

But the design goal is to not need them. A single-document write is **already atomic**, however deeply nested. If your documents are modelled so that one document is one unit of work, the invariants hold without any transaction. **Reaching for transactions routinely is a signal about the schema**, not a reason to tune the transaction.

The costs, which are real

- **A default time limit** — around a minute — after which the transaction aborts. Long-running work will simply fail
- **Snapshot state must be held** for the duration, adding cache pressure
- **Write conflicts abort and must be retried** by the application; the retry loop is your responsibility
- **Cross-shard transactions coordinate across nodes**, which is materially more expensive
- **The oplog entry is subject to size limits**, so very large transactions can fail outright

When they are genuinely right

- Moving value between two documents that cannot be merged into one — an account transfer
- Maintaining an invariant across genuinely separate aggregates
- Migrating a schema across collections in one step

Always use the driver's retry helper rather than a hand-written loop. `withTransaction` handles transient errors and commit-unknown results correctly — both of which are easy to get subtly wrong by hand.

12 · WIREDTIGER, MEMORY AND PERFORMANCE

STAFF why the working set fitting in cache is the single biggest performance factor

What WiredTiger provides

Document-level concurrency	Writers to different documents do not block each other — this is what makes concurrent write throughput acceptable
MVCC	Readers see a consistent snapshot without blocking writers
Compression	On by default for collections, with prefix compression for indexes. Trades CPU for a much smaller on-disk footprint
Journal	A write-ahead log flushed frequently; recovery replays it after a crash
Checkpoints	Periodic consistent snapshots of the data files

Cache sizing

The default is `max(50% of (RAM − 1 GB), 256 MB)`. The remainder is deliberately left for the operating system's page cache and for connection and query memory — which is why raising it toward 100% of RAM causes out-of-memory events rather than better performance.

Set it explicitly in containers. mongod may detect the **host's** total memory rather than the container's limit, size its cache against that, and be terminated by the platform. This is a very common and confusing failure in Kubernetes deployments.

The single biggest performance factor: does the working set fit in cache? The working set is the documents and index pages actually touched — not the whole database. While it fits, reads are memory-speed. Once it does not, every miss becomes a disk read, and performance does not degrade gently — it falls off a cliff as the miss rate climbs.

Diagnosing it

- **Cache hit ratio** from `serverStatus` — a falling ratio precedes almost every performance incident
- **Pages read into cache** rising steadily means the working set has outgrown it
- **Disk read IOPS and latency** climbing while query volume is flat
- **Index size versus cache size —** if indexes alone approach the cache, document lookups will always miss

What to do about it, in order

1. **Add indexes** so fewer documents are examined — often the whole fix
2. **Drop unused indexes**, which are consuming cache for nothing
3. **Project only needed fields**, so less is pulled through
4. **Reduce document size** — the subset pattern, or moving large blobs to object storage
5. **Archive cold data** so the working set shrinks
6. **Add RAM** — usually cheaper than the alternative below
7. **Shard**, which spreads the working set across machines. The last resort, not the first

Note the order. Six things come before sharding, and in most cases one of the first three resolves it. Sharding to fix a working-set problem that an index would have solved adds permanent operational cost for no benefit.

13 · SECURITY & BACKUP

the defaults are not enough

Access control

- **Enable authorisation.** Without it, anyone who can reach the port has full access — and unauthenticated MongoDB exposed to the internet is scanned for and compromised routinely
- **`bindIp` to specific interfaces.** Never expose it publicly, regardless of authentication
- **SCRAM** for password auth; **x.509 certificates**, **LDAP** or **Kerberos** for enterprise integration
- **Role-based access control** with built-in and custom roles — per database and per collection
- **Least privilege per application.** A reporting service needs read on one database, not cluster admin
- **TLS** for client and intra-cluster traffic

Encryption

- **At rest —** storage-engine or volume-level encryption
- **Client-side field-level encryption —** fields are encrypted by the driver, so the server never sees plaintext
- **Queryable encryption —** supports equality and range queries over encrypted fields, keeping them encrypted server-side

Backups

Filesystem snapshot	Fast and consistent if the journal is included. The usual production approach
`mongodump`	Logical export. Convenient at small scale, slow to restore at large scale
Point-in-time	A snapshot plus oplog replay to a chosen moment — the only thing that recovers from a bad write at a known time

Replication is not a backup. It copies a mistaken `deleteMany` to every member in milliseconds. Only a backup — one you have actually restored, and timed — protects against human error and application bugs.

14 · CONFIGURATION, DRIVERS AND OBSERVABILITY

the settings that change behaviour, what the application must get right, and what to watch

Server configuration that matters

Setting	Why
`wiredTiger.engineConfig.cacheSizeGB`	Leave at the default on a dedicated host; **set it explicitly in a container**, where host RAM may be detected instead of the limit
`security.authorization`	**enabled**. There is no acceptable reason to run without it
`net.bindIp`	Specific interfaces only. Never `0.0.0.0` on an untrusted network
`net.tls`	Client and intra-cluster encryption
`replication.oplogSizeMB`	Sets the replication window — size it against write rate and your worst maintenance window
`storage.journal`	Leave enabled. Disabling it trades durability for very little
`operationProfiling`	Profile slow operations above a threshold — the fastest route to the query causing latency
`net.maxIncomingConnections`	Bound it; connection storms are a real failure mode

Host settings matter as much. Use XFS for the data volume, disable transparent huge pages, raise file descriptor and process limits, and ensure the clock is synchronised — several of these are checked and warned about at startup, and the warnings are worth reading.

What the driver must get right

- **One client instance per application**, shared. Creating a client per request defeats pooling entirely and is a common source of connection exhaustion
- **`maxPoolSize`** sized against the server's connection limit divided by the number of application instances
- **Timeouts set —** connect, socket and server selection. Unbounded waits turn a slow server into a hung application
- **Retryable reads and writes enabled —** the driver retries once through a failover, which turns most elections into a latency blip rather than an error
- **Write and read concern set deliberately** per operation, not left implicit
- **Causally consistent sessions** where read-your-writes matters
- **The connection string lists all members**, or uses SRV, so the driver can discover the topology

Retryable writes deserve a mention in any interview. The driver retries a failed write once, and the server deduplicates it using a statement id — so a retry after a failover does not apply the write twice. It is what makes elections mostly invisible to the application.

What to monitor

Metric	Why
Replication lag	The size of your data-loss window right now, and the first thing to check in an incident
Oplog window	How long a member can be offline before needing a full resync
Cache hit ratio	Falls before almost every performance incident
Page faults / disk reads	The working set no longer fits
Connections current and available	Exhaustion saturates before CPU does
Queued operations	Read and write queues indicate saturation
Slow query count	From the profiler — usually one missing index
Index miss ratio	Queries scanning rather than seeking
Balancer activity	Chunk migrations competing with live traffic
Election count	Frequent elections mean instability, not healthy failover

The tools. `explain("executionStats")` for a single query; the **database profiler** for slow operations over time; `serverStatus` and `currentOp` for live state; `$indexStats` to find unused indexes; and `mongotop` and `mongostat` for a quick live view.

15 · PRODUCTION FAILURES — SYMPTOM, CAUSE AND FIX

what actually goes wrong, and the order in which you will meet it

Symptom	Cause	Fix
Queries suddenly slow, CPU high	A missing index causing a collection scan, or a plan change	Profiler then `explain`; add the index; check for `COLLSCAN`
Gradual slowdown over months	The working set has outgrown the cache	Cache hit ratio; add indexes, reduce document size, archive, add RAM
Writes fail after a period of growth	A document approaching the 16 MB limit — almost always an unbounded array	Restructure to reference the growing collection; the bucket pattern
One shard hot, others idle	A monotonic or low-cardinality shard key	Hashed or compound key; resharding if it is severe enough to justify it
Data missing after a failover	Writes acknowledged with `w: 1` were rolled back	Expected behaviour. Use `w: "majority"`; check the rollback files

Symptom	Cause	Fix
Secondary needs a full resync	It fell behind the oplog window during maintenance	Increase oplog size; monitor the window as a metric
Connection errors under load	Pool too small, or a client created per request	One shared client; size `maxPoolSize` against the server limit
mongod killed by the platform	Cache sized against host RAM inside a container	Set `cacheSizeGB` explicitly below the container limit
Stale reads reported by users	Reading from secondaries during replication lag	Causally consistent sessions, or read from the primary for those paths
Latency spikes at intervals	Balancer migrating chunks, or checkpoint pressure	Schedule a balancing window; review write volume and disk throughput
Aggregation fails outright	A stage exceeded its memory limit	Filter earlier; add a supporting index; enable disk use knowing it is slower

What breaks first

#	Limit
1	**A missing index.** By a wide margin the most common cause of a sudden incident, and the cheapest to fix
2	**Working set exceeding cache**, which arrives gradually and then sharply
3	**Connection pool exhaustion**, which saturates before CPU
4	**A hot shard** from a poor shard key — structural, and expensive to correct
5	**Single-primary write throughput** — the real ceiling, and the last one reached

Note the order. The limit people design for — write throughput — is last. The two that actually cause incidents are an index and a memory budget, both of which are fixable in an afternoon if they are being measured.

Degradation

- **Primary unavailable —** writes fail for the seconds until an election completes. Retryable writes make this mostly invisible if the driver is configured for it
- **A secondary lost —** no user impact, but a majority write concern now has less margin
- **Config servers unavailable —** a sharded cluster cannot route. Treat their availability as critical
- **mongos unavailable —** run several; they are stateless and cheap

16 · PRODUCTION CHECKLIST

before an application depends on it

- ☐ **Schema modelled from queries**, not from entities
- ☐ **No unbounded arrays** — every embedded array has a known ceiling
- ☐ **Document size monitored** against the 16 MB limit
- ☐ **Embed-versus-reference decided per relationship**, by cardinality
- ☐ **Duplicated fields have an update plan** and a stated staleness tolerance
- ☐ **Application validates the schema**, or JSON schema validation is enabled
- ☐ **Indexes exist for every frequent query**, verified with `explain`
- ☐ **Compound index order follows ESR**
- ☐ **No `COLLSCAN`** on any hot path
- ☐ **Unused indexes identified and dropped**
- ☐ **Indexes and working set fit in cache**, with headroom
- ☐ **Cache hit ratio monitored** and alerted on
- ☐ **`cacheSizeGB` set explicitly** if running in a container
- ☐ **Odd number of voting members**, in separate failure domains
- ☐ **No arbiter** where majority write concern matters
- ☐ **Write concern set deliberately** — `w: "majority"` for anything durable
- ☐ **Read concern set to match**; a majority write with a local read is not safe
- ☐ **Rollback behaviour understood** and accepted, or prevented
- ☐ **Read preference chosen per query**, with staleness accepted where used
- ☐ **Causal sessions used** where read-your-writes matters
- ☐ **Oplog sized** against write rate and worst maintenance window
- ☐ **Replication lag and oplog window alerted on**
- ☐ **Failover tested**, including that the application reconnects
- ☐ **Shard key chosen from the five most frequent queries**
- ☐ **Shard key has cardinality, even frequency and is non-monotonic**
- ☐ **Scatter-gather queries identified** and accepted or eliminated
- ☐ **Balancer window scheduled** if traffic has a clear peak
- ☐ **Config servers treated as critical**, replicated and monitored
- ☐ **Several mongos instances**, close to the application
- ☐ **Transactions used sparingly**, with the driver's retry helper
- ☐ **Aggregations filter early**; memory limits understood
- ☐ **One shared client per application**; pool sized against the server limit
- ☐ **Driver timeouts set**; retryable reads and writes enabled
- ☐ **Authorisation enabled**; never bound to a public interface
- ☐ **TLS in transit**; encryption at rest where required
- ☐ **Least-privilege roles per application**
- ☐ **Backups automated and off-host**, with a restore tested and timed
- ☐ **Point-in-time recovery available** where a bad write must be undone
- ☐ **Slow query profiler enabled** and reviewed regularly
- ☐ **Host settings applied —** filesystem, THP, ulimits, clock sync

The one that summarises the sheet: **name what one document represents, whether any array in it can grow without bound, what the shard key is and which queries filter on it, and what write concern the application actually uses.** Those four answers reveal almost every serious problem — and the last is very often "whatever the default is", which is worth discovering before a failover rather than after one.